

Name der Hochschule
Fakultät / Studiengang

Vorlesung: Software-Entwurf / Entwurfsmuster
Dozent/in: Prof. Dr. Name

Das Decorator-Pattern

Ein Strukturmuster zur dynamischen Erweiterung von Objekten

Projektausarbeitung

Vorname Nachname
Matrikelnummer: 000000

1. Oktober 2026

Zusammenfassung

Das Decorator-Pattern ist ein objektbasiertes Strukturmuster aus dem Katalog der „Gang of Four“. Es erlaubt, einem Objekt zur Laufzeit zusätzliche Verantwortlichkeiten zuzuordnen, indem es von Hüllobjekten mit identischer Schnittstelle umgeben wird. Die vorliegende Ausarbeitung stellt Zweck, Struktur und Teilnehmer des Musters vor, erläutert es anhand von UML-Diagrammen sowie eines abstrakten und zweier konkreter Codebeispiele in Java und grenzt es gegenüber der Dekorator-Syntax in Python ab. Anschließend werden Vor- und Nachteile, Kritik und Alternativen diskutiert und das Muster von verwandten Mustern wie Adapter, Proxy, Composite, Strategy und Chain of Responsibility abgegrenzt. Das Ergebnis: Der Decorator ist ein bewährtes Mittel gegen Klassenexplosion und überladene Basisklassen, führt aber auch zu vielen kleinen Objekten und einer schwerer nachvollziehbaren Laufzeitstruktur.

Inhaltsverzeichnis

Zusammenfassung	ii
Abbildungsverzeichnis	iv
Tabellenverzeichnis	v
Listingverzeichnis	vi
Abkürzungsverzeichnis	vii
1 Einleitung	1
1.1 Motivation	1
1.2 Zielsetzung und Aufbau	1
2 Grundlagen: Entwurfsmuster und Entwurfsprinzipien	1
3 Das Decorator-Pattern	2
3.1 Zweck und Motivation	2
3.2 Struktur und Teilnehmer	2
3.3 Zusammenspiel	3
3.4 Abstraktes Codebeispiel	3
4 Anwendungsbeispiele	4
4.1 Getränkebestellung in Java	4
4.2 Die Java-Ein-/Ausgabe-Bibliothek	5
4.3 Abgrenzung: Dekorator-Syntax in Python	6
5 Bewertung	7
5.1 Vorteile	7
5.2 Nachteile	7
5.3 Kritik und Alternativen	8
6 Verwandte Muster und Abgrenzung	8
7 Fazit	9
Literatur	10

Abbildungsverzeichnis

1	Klassendiagramm des Decorator-Patterns (nach [GHJV94]). Gestrichelte Pfeile: Realisierung der Schnittstelle, Raute: Aggregation der umhüllten Komponente, durchgezogene Pfeile: Vererbung.	3
2	Objektdiagramm der Verschachtelung <code>Sugar(Milk(Espresso))</code> : Der Aufruf wird von außen nach innen delegiert, das Ergebnis (2,39) entsteht auf dem Rückweg.	3
3	Verkettung von Decorators in <code>java.io</code> : Leseanfragen werden bei Bedarf von links nach rechts delegiert.	6

Tabellenverzeichnis

1	Rollen des Decorator-Patterns und ihre Entsprechungen in den Beispielen dieser Arbeit	2
2	Abgrenzung des Decorators von verwandten Mustern (nach [GHJV94])	9

Listingverzeichnis

1	Abstrakte Implementierung des Decorator-Patterns in Java	4
2	Getränkebestellung mit dem Decorator-Pattern (Datei <code>Kaffeebar.java</code>) . . .	5
3	Verschachtelte Stream-Decorators (innerhalb einer Methode mit <code>throws IOException</code>)	6
4	Reihenfolge der Decorators: erst komprimieren, dann verschlüsseln (<code>cipher</code> sei ein initialisiertes <code>javax.crypto.Cipher</code>)	6
5	Python-Funktionsdekorator zur Zeitmessung	7

Abkürzungsverzeichnis

AOP Aspektorientierte Programmierung

GoF Gang of Four (Gamma, Helm, Johnson, Vlissides)

I/O Input/Output (Ein-/Ausgabe)

JDK Java Development Kit

LSP Liskovsches Substitutionsprinzip

OCP Open-Closed-Prinzip

OOP Objektorientierte Programmierung

PEP Python Enhancement Proposal

SRP Single-Responsibility-Prinzip

UML Unified Modeling Language

1 Einleitung

1.1 Motivation

Objektorientierte Software bündelt Daten und zugehörige Funktionen in Objekten und dient dazu, Anwendungen in zusammenarbeitende, überschaubare Bausteine zu strukturieren.

Objektorientierte Software soll erweiterbar sein, ohne dass bestehender, getesteter Code verändert werden muss. Das klassische Mittel zur Erweiterung ist die Vererbung. Sie ist jedoch statisch: Das Verhalten einer Klasse wird zur Übersetzungszeit festgelegt, und jede Kombination optionaler Eigenschaften erfordert eine eigene Unterklasse. Schon bei n unabhängig kombinierbaren Zusatzfunktionen wären bis zu 2^n Varianten zu modellieren. Dieses Problem wird als *Klassenenexplosion* bezeichnet. Außerdem koppelt Vererbung Unter- und Oberklasse eng aneinander. Änderungen an der Oberklasse können dadurch das Verhalten bestehender Unterklassen unbeabsichtigt beeinträchtigen; dies wird als Problem der *fragilen Basisklasse* bezeichnet [MS98, Blo18].

Das Decorator-Pattern löst dieses Problem durch Komposition statt Vererbung: Zusätzliches Verhalten wird in kleine, austauschbare Hüllobjekte ausgelagert, die zur Laufzeit beliebig kombiniert werden können [GHJV94, S. 175–184].

1.2 Zielsetzung und Aufbau

Ziel der Arbeit ist eine kompakte, wissenschaftlich fundierte Darstellung des Patterns. Abschnitt 2 ordnet das Muster in die Geschichte der Entwurfsmuster und die zugrunde liegenden Entwurfsprinzipien ein. Abschnitt 3 behandelt Zweck, Struktur und Verhalten samt abstraktem Codebeispiel. Abschnitt 4 zeigt konkrete Anwendungen in Java und Python. Abschnitt 5 bewertet das Muster kritisch, Abschnitt 6 grenzt es von verwandten Mustern ab, und Abschnitt 7 fasst die Ergebnisse zusammen.

2 Grundlagen: Entwurfsmuster und Entwurfsprinzipien

Entwurfsmuster beschreiben bewährte Lösungsansätze für wiederkehrende Entwurfsprobleme und dienen als anpassbare Vorlagen, nicht als fertiger Programmcode.

Der Begriff des Entwurfsmusters stammt ursprünglich aus der Architektur: Alexander et al. beschreiben wiederkehrende Lösungen für wiederkehrende Probleme in einer „Mustersprache“ [AIS77]. Beck und Cunningham übertrugen diese Idee 1987 auf die objektorientierte Programmierung (OOP) [BC87]. Breite Wirkung erzielte der 1994 erschienene Katalog von 23 Mustern von Gamma, Helm, Johnson und Vlissides, der als „Gang of Four“ (GoF) bekannt wurde [GHJV94]. Er unterteilt die Muster nach ihrem Zweck in Erzeugungs-, Struktur- und Verhaltensmuster. Der Decorator gehört zu den *Strukturmustern*, die beschreiben, wie Klassen und Objekte zu größeren Strukturen zusammengesetzt werden.

Drei Entwurfsprinzipien sind für das Verständnis des Decorators zentral:

- **Komposition vor Vererbung.** Die GoF empfehlen, gegen eine Schnittstelle zu programmieren und Objektkomposition der Klassenvererbung vorzuziehen [GHJV94]. Bloch begründet dies ausführlich und nennt Wrapper-Klassen mit Weiterleitung (Delegation) als Alternative zur fehleranfälligen Vererbung [Blo18].

Tabelle 1: Rollen des Decorator-Patterns und ihre Entsprechungen in den Beispielen dieser Arbeit

Rolle	Aufgabe	Getränkebeispiel	java.io
Component	gemeinsame Schnittstelle	Beverage	InputStream
ConcreteComponent	zu erweiterndes Objekt	Espresso	FileInputStream
Decorator	hält Referenz auf Component, leitet Aufrufe weiter	CondimentDecorator	FilterInputStream
ConcreteDecorator	fügt konkretes Verhalten hinzu	Milk, Sugar	BufferedInputStream, DataInputStream

- **Open-Closed-Prinzip (OCP).** Software-Einheiten sollen offen für Erweiterung, aber geschlossen für Änderung sein [Mey88, Mar03].
- **Single-Responsibility-Prinzip (SRP).** Jede Klasse soll genau einen Grund zur Änderung haben [Mar03]. Beim Decorator kapselt jede Hülle genau eine Zusatzaufgabe.

3 Das Decorator-Pattern

3.1 Zweck und Motivation

Nicht jedes Objekt einer Klasse benötigt dieselben Zusatzfunktionen. Der Decorator ermöglicht es, einzelne Objekte gezielt zu erweitern, ohne dafür jeweils eine neue Unterklasse anzulegen.

Nach [GHJV94, S. 175] besteht der Zweck des Patterns darin, einem Objekt dynamisch zusätzliche Verantwortlichkeiten zuzuordnen; Decorators bieten damit eine flexible Alternative zur Unterklassenbildung. Das Muster ist auch unter dem Namen *Wrapper* bekannt. Das Standardbeispiel der GoF ist eine GUI-Komponente (`TextView`), die wahlweise einen Rahmen oder Scrollbalken erhalten soll. Statt für jede Kombination eine Unterklasse anzulegen, wird die Komponente in einen `BorderDecorator` und/oder `ScrollDecorator` eingepackt.

Das Muster ist nach [GHJV94] anwendbar, wenn

- Verantwortlichkeiten einzelnen Objekten dynamisch und transparent hinzugefügt oder wieder entzogen werden sollen, ohne andere Objekte zu beeinflussen,
- eine Erweiterung durch Unterklassen unpraktikabel ist, etwa wegen einer Explosion der Klassenzahl, oder die Klasse nicht ableitbar ist (z. B. `final`).

3.2 Struktur und Teilnehmer

Das Muster besteht aus einer gemeinsamen Schnittstelle, einer konkreten Komponente und Decorators, die diese Komponente um zusätzliches Verhalten erweitern.

Abbildung 1 zeigt das Klassendiagramm des Musters in Unified Modeling Language (UML)-Notation, Tabelle 1 ordnet die Rollen den Beispielen dieser Arbeit zu.

Die zentrale Idee: Der **Decorator** implementiert dieselbe Schnittstelle wie die umhüllte **Component** und hält eine Referenz auf sie. Clients können dekorierte und undekorierte Objekte über dieselbe Schnittstelle verwenden (Transparenz). Ein **ConcreteDecorator** ruft die

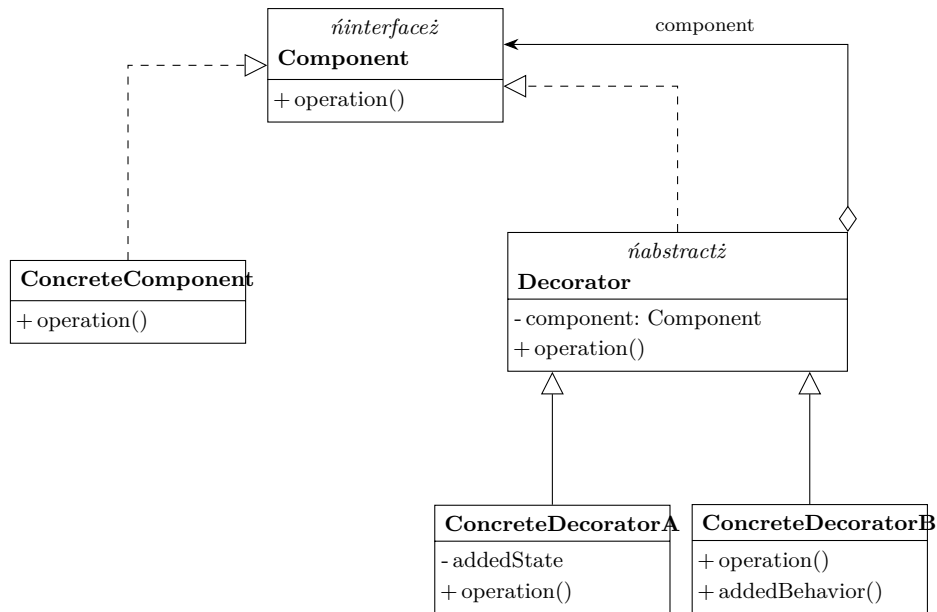


Abbildung 1: Klassendiagramm des Decorator-Patterns (nach [GHJV94]). Gestrichelte Pfeile: Realisierung der Schnittstelle, Raute: Aggregation der umhüllten Komponente, durchgezogene Pfeile: Vererbung.

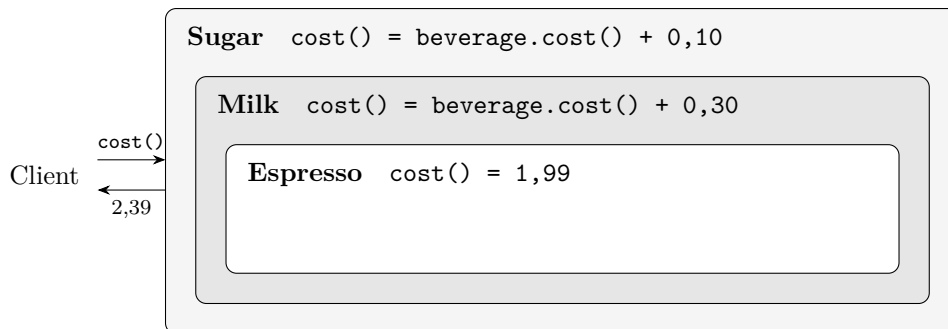


Abbildung 2: Objektdiagramm der Verschachtelung `Sugar(Milk(Espresso))`: Der Aufruf wird von außen nach innen delegiert, das Ergebnis (2,39) entsteht auf dem Rückweg.

Operation der umhüllten Komponente auf und ergänzt davor und/oder danach eigenes Verhalten. Da Decorators selbst **Components** sind, lassen sie sich beliebig tief verschachteln.

3.3 Zusammenspiel

Abbildung 2 veranschaulicht die Laufzeitstruktur am Beispiel einer Getränkebestellung: Ein Aufruf von `cost()` am äußersten Decorator wird schrittweise nach innen delegiert; die Ergebnisse werden auf dem Rückweg aufaddiert. Die Struktur entspricht damit einer Schichtung wie bei einer Zwiebel.

3.4 Abstraktes Codebeispiel

Listing 1 zeigt das Muster in abstrakter Form in Java. Die Ausgabe `B(A(Basis))` macht die Reihenfolge der Verschachtelung sichtbar: Der zuletzt angelegte Decorator ist der äußerste und wird zuerst durchlaufen.

```
1 interface Component {
2     String operation();
3 }
4
5 class ConcreteComponent implements Component {
6     public String operation() { return "Basis"; }
7 }
8
9 abstract class Decorator implements Component {
10     protected final Component component;
11
12     protected Decorator(Component component) {
13         this.component = component;
14     }
15
16     public String operation() {           // Standard: nur weiterleiten
17         return component.operation();
18     }
19 }
20
21 class ConcreteDecoratorA extends Decorator {
22     ConcreteDecoratorA(Component c) { super(c); }
23
24     public String operation() {
25         return "A(" + super.operation() + ")";
26     }
27 }
28
29 class ConcreteDecoratorB extends Decorator {
30     ConcreteDecoratorB(Component c) { super(c); }
31
32     public String operation() {
33         return "B(" + super.operation() + ")";
34     }
35 }
36
37 public class Demo {
38     public static void main(String[] args) {
39         Component c = new ConcreteDecoratorB(
40             new ConcreteDecoratorA(new ConcreteComponent()));
41         System.out.println(c.operation()); // Ausgabe: B(A(Basis))
42     }
43 }
```

Listing 1: Abstrakte Implementierung des Decorator-Patterns in Java

4 Anwendungsbeispiele

4.1 Getränkebestellung in Java

Das Standardbeispiel aus der Lehrbuchliteratur [FR20] modelliert eine Kaffeebar: Ein Getränk kann mit beliebigen Zutaten (Milch, Zucker, Sahne, ...) bestellt werden, und der Preis ergibt sich aus Grundpreis und Zuschlägen. Mit Vererbung wären bereits für drei Zutaten

sieben zusätzliche Unterklassen je Getränk nötig, mit dem Decorator genügt eine Klasse pro Zutat. Listing 2 zeigt eine vollständige, lauffähige Umsetzung.

```

1  import java.util.Locale;
2  import java.util.Objects;
3
4  interface Beverage {                               // Component
5      String getDescription();
6      double cost();
7  }
8
9  class Espresso implements Beverage {               // ConcreteComponent
10     public String getDescription() { return "Espresso"; }
11     public double cost() { return 1.99; }
12 }
13
14 abstract class CondimentDecorator implements Beverage { // Decorator
15     protected final Beverage beverage;
16
17     protected CondimentDecorator(Beverage beverage) {
18         this.beverage = Objects.requireNonNull(beverage);
19     }
20 }
21
22 class Milk extends CondimentDecorator {             // ConcreteDecorator
23     Milk(Beverage b) { super(b); }
24     public String getDescription() { return beverage.getDescription() + ", Milch"; }
25     public double cost() { return beverage.cost() + 0.30; }
26 }
27
28 class Sugar extends CondimentDecorator {            // ConcreteDecorator
29     Sugar(Beverage b) { super(b); }
30     public String getDescription() { return beverage.getDescription() + ", Zucker"; }
31     public double cost() { return beverage.cost() + 0.10; }
32 }
33
34 public class Kaffeebar {
35     public static void main(String[] args) {
36         Beverage order = new Sugar(new Milk(new Espresso()));
37         System.out.println(order.getDescription() + ": "
38             + String.format(Locale.GERMANY, "%.2f EUR", order.cost()));
39         // Ausgabe: Espresso, Milch, Zucker: 2,39 EUR
40     }
41 }

```

Listing 2: Getränkebestellung mit dem Decorator-Pattern (Datei Kaffeebar.java)

Für reale Preisberechnungen sollte statt `double` ein exakter Typ wie `BigDecimal` verwendet werden. Hier dient `double` nur der Übersichtlichkeit.

4.2 Die Java-Ein-/Ausgabe-Bibliothek

Das Decorator-Pattern wird nicht nur in Lehrbeispielen, sondern auch in verbreiteten Bibliotheken eingesetzt.

Ein prominentes Beispiel aus der Standardbibliothek ist das Paket `java.io`. Die Klasse

`FilterInputStream` umhüllt einen anderen `InputStream` und leitet Aufrufe an ihn weiter; ihre Unterklassen (z. B. `BufferedInputStream`, `DataInputStream`) ergänzen Pufferung bzw. das Lesen typisierter Daten [ORA-FIS]. Die Ein-/Ausgabe (I/O) lässt sich so flexibel aus kleinen Bausteinen zusammensetzen. Abbildung 3 zeigt die Schichtung der beteiligten Streams; Listing 3 veranschaulicht ihre Verwendung beim Lesen typisierter Messwerte.

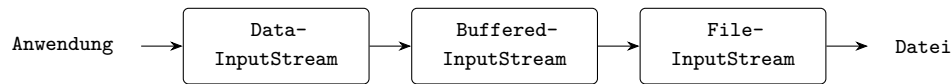


Abbildung 3: Verkettung von Decorators in `java.io`: Leseanfragen werden bei Bedarf von links nach rechts delegiert.

```

1 try (DataInputStream in = new DataInputStream(
2     new BufferedInputStream(
3         new FileInputStream("messwerte.bin")))) {
4     int anzahl = in.readInt();
5     double summe = 0;
6     for (int i = 0; i < anzahl; i++) {
7         summe += in.readDouble();
8     }
9     System.out.println("Mittelwert: " + summe / anzahl);
10 }
  
```

Listing 3: Verschachtelte Stream-Decorators (innerhalb einer Methode mit `throws IOException`)

Das Beispiel zeigt auch, dass die *Reihenfolge* der Decorators relevant sein kann: Beim Schreiben wird die äußerste Hülle zuerst durchlaufen. Werden Daten komprimiert und verschlüsselt, so muss erst komprimiert und dann verschlüsselt werden (Listing 4), da verschlüsselte Daten sich kaum noch komprimieren lassen.

```

1 OutputStream out = new GZIPOutputStream(
2     new CipherOutputStream(
3         new FileOutputStream("daten.bin"), cipher));
  
```

Listing 4: Reihenfolge der Decorators: erst komprimieren, dann verschlüsseln (`cipher` sei ein initialisiertes `javax.crypto.Cipher`)

Nicht jede Klasse in `java.io`, die etwas umhüllt, ist ein Decorator. Der `InputStreamReader`, der einen Byte-Strom in einen Zeichen-Strom überführt, ändert die Schnittstelle und ist daher ein *Adapter* (vgl. Abschnitt 6). Weitere Decorator-artige Wrapper im Java Development Kit (JDK) sind etwa `Collections.unmodifiableList` und `Collections.synchronizedList`, die eine Liste um Schreibschutz bzw. Synchronisation ergänzen.

4.3 Abgrenzung: Dekorator-Syntax in Python

Python bietet mit der `@`-Syntax eine Spracheigenschaft, die ebenfalls „Decorator“ heißt. Sie wurde mit Python Enhancement Proposal (PEP) 318 in Python 2.4 eingeführt [PEP318]. Ein solcher Decorator ist eine Funktion höherer Ordnung, die eine Funktion (oder Klasse) bei ihrer Definition durch eine umhüllende Variante ersetzt (Listing 5). Die Idee – Verhalten transparent um einen bestehenden Aufruf zu legen – ist verwandt. Das GoF-Pattern arbeitet jedoch auf *Objekten* und deren Schnittstelle zur Laufzeit, während die Python-Syntax eine

Kurzschreibweise für das Ersetzen von Funktionen ist. Die Dienstfunktion `functools.wraps` übernimmt Metadaten wie Name und Docstring in den Wrapper [PYFT].

```
1 import functools
2 import time
3
4 def timed(func):
5     @functools.wraps(func)
6     def wrapper(*args, **kwargs):
7         start = time.perf_counter()
8         try:
9             return func(*args, **kwargs)
10        finally:
11            dauer = time.perf_counter() - start
12            print(f"{func.__name__}: {dauer:.4f} s")
13        return wrapper
14
15 @timed
16 def quadratsumme(n):
17     return sum(i * i for i in range(n))
18
19 quadratsumme(1_000_000)
```

Listing 5: Python-Funktionsdekorator zur Zeitmessung

5 Bewertung

5.1 Vorteile

- **Flexibilität:** Verhalten wird zur Laufzeit zugefügt, entfernt und kombiniert. Das ist flexibler als statische Vererbung [GHJV94].
- **Vermeidung von Klassenexplosion und überladenen Basisklassen:** Mit n Zusatzfunktionen genügen n Decorator-Klassen statt bis zu 2^n Unterklassen. Funktionsreiche Klassen weit oben in der Hierarchie werden vermieden [GHJV94].
- **SRP und OCP:** Jeder Decorator hat genau eine Aufgabe, bestehender Code bleibt unverändert [Mar03].
- **Transparenz:** Clients arbeiten gegen die Schnittstelle `Component` und kennen die Dekoration nicht.

5.2 Nachteile

- **Viele kleine Objekte und Klassen:** Systeme werden zwar leicht anpassbar, aber schwer zu überblicken und zu debuggen, da Aufrufe viele Schichten durchlaufen [GHJV94].
- **Objektidentität:** Ein Decorator ist nicht identisch mit der umhüllten Komponente. Identitätsvergleiche (`==`) und Typprüfungen (`instanceof`) liefern unerwartete Ergebnisse [GHJV94].
- **Reihenfolgeabhängigkeit:** Wie in Listing 4 gezeigt, kann die Verkettung das Ergebnis verändern.

- **Schwierige Entfernung:** Einzelne Decorators aus der Mitte einer Kette zu entfernen ist umständlich.
- **Boilerplate:** Bei breiten Schnittstellen muss jeder Decorator viele Methoden bloß weiterleiten. Die GoF raten deshalb, die **Component**-Schnittstelle schlank zu halten [GHJV94].
- **Vertragstreue:** Damit Decorators wirklich austauschbar sind, müssen sie das Liskovsche Substitutionsprinzip (LSP) einhalten [LW94].

Für die Praxis sollten daher nicht nur einzelne Decorators, sondern auch ihre Kombinationen und Reihenfolgen getestet werden.

5.3 Kritik und Alternativen

Entwurfsmuster sind nicht unumstritten. Norvig argumentiert, dass 16 der 23 GoF-Muster in dynamischen Sprachen wie Lisp oder Dylan unsichtbar oder deutlich einfacher sind, weil die Sprache die nötigen Mittel bereits bietet [Nor96]. Muster seien insofern auch Hinweise auf Defizite einer Programmiersprache. Für den Decorator treffen ähnliche Überlegungen zu: Moderne Sprachen und Werkzeuge bieten Alternativen, die die Weiterleitungs-Boilerplate verringern oder das Muster ersetzen können:

- **Sprachgestützte Delegation:** Kotlin erlaubt über das Schlüsselwort `by`, eine Schnittstelle an ein Objekt zu delegieren, ohne die Methoden einzeln auszuschreiben [KOTLIN].
- **Dynamische Proxies:** In Java kann `java.lang.reflect.Proxy` Weiterleitungen zur Laufzeit erzeugen [ORA-PROXY].
- **Mixins und Traits:** Sie fügen Verhalten per Klassenkomposition hinzu, allerdings statisch pro Klasse und nicht pro Objekt [BC90].
- **Aspektorientierte Programmierung (AOP):** Querschnittsbelange wie Logging oder Transaktionen werden zentral in Aspekten gebündelt statt in Hüllklassen [Kic97].

Diese Alternativen lösen jeweils Teilaspekte; sie bieten jedoch nicht alle die Möglichkeit, Verhalten pro Objekt und zur Laufzeit zu wählen. Der Decorator bleibt deshalb dort sinnvoll, wo genau dies verlangt ist.

6 Verwandte Muster und Abgrenzung

Decorator, Adapter und Proxy sind Wrapper-Muster: Alle umhüllen ein Objekt und leiten Aufrufe weiter, verfolgen aber unterschiedliche Absichten [GHJV94]. Tabelle 2 fasst die Unterschiede zusammen. Die GoF beschreiben den Gegensatz zu Strategy anschaulich: Ein Decorator ändert die „Haut“ eines Objekts, eine Strategy sein „Innenleben“ [GHJV94].

Tabelle 2: Abgrenzung des Decorators von verwandten Mustern (nach [GHJV94])

Muster	Gemeinsamkeit mit Decorator	Unterschied
Adapter	umhüllt ein Objekt	ändert die Schnittstelle; der Decorator behält sie bei
Proxy	gleiche Schnittstelle, umhüllt ein Objekt	steuert den Zugriff (z. B. Lazy Loading, Schutz, Fernzugriff); der Decorator fügt Verhalten hinzu
Composite	rekursive Objektstruktur, gleiche Schnittstelle	Composite hat viele Kinder und fasst Objekte zusammen; der Decorator hat genau eine Komponente und erweitert sie
Strategy	verändert das Verhalten eines Objekts	Strategy tauscht das „Innenleben“ aus (Algorithmus), der Decorator ändert die „Hülle“; Strategy erfordert Erweiterungspunkte im Objekt
Chain of Responsibility	Verkettung von Objekten	dort kann ein Glied die Anfrage abschließend behandeln und die Kette abbrechen; ein Decorator reicht typischerweise weiter
Template Method	Erweiterung von Verhalten	Variation per Vererbung statt per Komposition

In der Praxis treten die Muster oft kombiniert auf: So lässt sich ein Composite mit Decorators um Zusatzverhalten für einzelne Knoten ergänzen, und ein Proxy kann strukturell wie ein Decorator aufgebaut sein. Die Abgrenzung erfolgt dann über die *Absicht*, nicht über die Struktur [GHJV94].

7 Fazit

Das Decorator-Pattern ist ein bewährtes Mittel, Objekten zur Laufzeit Verantwortlichkeiten hinzuzufügen, ohne Klassen zu verändern oder eine Vielzahl von Unterklassen anzulegen. Es setzt die Prinzipien „Komposition vor Vererbung“, OCP und SRP konsequent um und ist in der Praxis, etwa in `java.io`, weit verbreitet. Dem stehen eine unübersichtliche Laufzeitstruktur, Identitätsprobleme, Reihenfolgeabhängigkeiten und Weiterleitungs-Boilerplate gegenüber. Sprachmittel wie Delegation, dynamische Proxies, Mixins oder AOP können je nach Kontext Teilaufgaben des Musters übernehmen. Empfehlenswert ist der Decorator vor allem bei schlanken Schnittstellen und wenn Zusatzverhalten pro Objekt und zur Laufzeit kombinierbar sein soll. Bei breiten Schnittstellen und statisch feststehender Funktionalität sind Alternativen häufig einfacher.

Literatur

- [AIS77] Alexander, C.; Ishikawa, S.; Silverstein, M.: *A Pattern Language: Towns, Buildings, Construction*. Oxford University Press, New York, 1977.
- [BC87] Beck, K.; Cunningham, W.: Using Pattern Languages for Object-Oriented Programs. Beitrag zum OOPSLA'87 Workshop on Specification and Design for Object-Oriented Programming, 1987.
- [Blo18] Bloch, J.: *Effective Java*. 3. Aufl., Addison-Wesley, Boston, 2018 (insb. Item 18: Favor composition over inheritance).
- [BC90] Bracha, G.; Cook, W.: Mixin-based inheritance. In: *Proceedings of OOPSLA/E-COOP '90*, ACM, 1990, S. 303–311.
- [FR20] Freeman, E.; Robson, E.: *Head First Design Patterns*. 2. Aufl., O'Reilly Media, Sebastopol, 2020.
- [GHJV94] Gamma, E.; Helm, R.; Johnson, R.; Vlissides, J.: *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley, Reading, 1994 (Kapitel 4: Decorator, S. 175–184).
- [Kic97] Kiczales, G.; Lamping, J.; Mendhekar, A.; Maeda, C.; Lopes, C.; Loingtier, J.-M.; Irwin, J.: Aspect-Oriented Programming. In: *ECOOP '97*, LNCS 1241, Springer, 1997, S. 220–242.
- [KOTLIN] JetBrains: Kotlin Documentation – Delegation. <https://kotlinlang.org/docs/delegation.html>, abgerufen am 01.10.2026.
- [LW94] Liskov, B. H.; Wing, J. M.: A Behavioral Notion of Subtyping. *ACM Transactions on Programming Languages and Systems* 16(6), 1994, S. 1811–1841.
- [Mar03] Martin, R. C.: *Agile Software Development: Principles, Patterns, and Practices*. Prentice Hall, Upper Saddle River, 2003.
- [Mey88] Meyer, B.: *Object-Oriented Software Construction*. Prentice Hall, 1988 (2. Aufl. 1997).
- [MS98] Mikhajlov, L.; Sekerinski, E.: A Study of the Fragile Base Class Problem. In: *ECOOP '98*, LNCS 1445, Springer, 1998, S. 355–382.
- [Nor96] Norvig, P.: Design Patterns in Dynamic Programming. Vortrag auf der Object World, Boston, 1996.
- [ORA-FIS] Oracle: Java SE API Documentation – `java.io.FilterInputStream`. <https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/io/FilterInputStream.html>, abgerufen am 01.10.2026.
- [ORA-PROXY] Oracle: Java SE API Documentation – `java.lang.reflect.Proxy`. <https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/lang/reflect/Proxy.html>, abgerufen am 01.10.2026.

- [PEP318] Smith, K. D.; Jewett, J.; Montanaro, S.; Baxter, A.: PEP 318 – Decorators for Functions and Methods. Python Software Foundation, 2003. <https://peps.python.org/pep-0318/>, abgerufen am 01.10.2026.
- [PYFT] Python Software Foundation: `functools` – Higher-order functions and operations on callable objects. Python Documentation. <https://docs.python.org/3/library/functools.html>, abgerufen am 01.10.2026.